

DEPLOYMENT GUIDE

Deploying the Sonar Auto-Fix Agent to Google Cloud

Step-by-step: standing up SonarQube on a Compute Engine VM, then shipping the agent to Cloud Run — both from the command line and via GitHub Actions.

Piece 1 — SonarQube

A long-running server on a Compute Engine VM, via Docker Compose.

Piece 2 — The Agent

A run-to-completion Cloud Run Job, built from this repo's Dockerfile.

remediation-agent — sonar_autofix_agent

Generated 2026-08-24

Contents

1. Prerequisites

2. Part A — Set up SonarQube on a Compute Engine VM

2.1 Create the VM

2.2 Open the firewall

2.3 Prep the host

2.4 Bring the stack up

2.5 First login, token, and the initial-scan gotcha

2.6 Branch analysis on Community Build

2.7 Stopping/starting the VM to save cost

3. Part B — Deploy the Agent from the Command Line

3.1 Build and push the image

3.2 Store secrets

3.3 Create the Cloud Run Job

3.4 Run it and watch it

3.5 Redeploying an update

4. Part C — Deploy the Agent via GitHub Actions

4.1 What the workflow does

4.2 One-time GCP-side setup (Workload Identity Federation)

4.3 One-time GitHub-side setup

4.4 Triggering a deploy

5. Networking: Cloud Run reaching the SonarQube VM

6. Optional: scheduled runs

7. What this costs

8. Recap — automated vs. still manual

Appendix — files referenced in this guide

STEP 1

Prerequisites

Everything below uses placeholders in `ALL_CAPS` — swap in your own values. Commands are `gcloud` / `docker`, run from your own machine.

```
# authenticate once
gcloud auth login
gcloud config set project PROJECT_ID
```

You'll also need, from your existing `.env` (never commit or paste these anywhere public):

- `GOOGLE_API_KEY` — Gemini access
- A GitHub token with `Contents: Read & write` on the target repo(s)

Two independent pieces get built in this guide, in order: **SonarQube** (a long-running server, Compute Engine VM) and **the agent** (a run-to-completion job, deployed as a Cloud Run *Job* — not a Service, since it fetches, fixes, verifies, and exits, exactly what a Job is for).

Set up SonarQube on a Compute Engine VM

2.1 Create the VM

```
gcloud services enable compute.googleapis.com

gcloud compute instances create sonarqube-vm \
  --zone=ZONE \
  --machine-type=e2-standard-2 \
  --image-family=ubuntu-2204-lts \
  --image-project=ubuntu-os-cloud \
  --boot-disk-size=30GB \
  --tags=sonarqube
```

`e2-standard-2` (2 vCPU / 8 GB) is SonarQube's realistic minimum — it runs an embedded Elasticsearch alongside the web/compute-engine processes and is noticeably unhappy on anything smaller. Check the [GCP pricing calculator](#) for a current estimate for your region rather than trusting a fixed number — pricing changes.

2.2 Open the firewall

```
gcloud compute firewall-rules create allow-sonarqube \
  --allow=tcp:9000 \
  --target-tags=sonarqube \
  --source-ranges=YOUR_IP/32
```

Start scoped to your own IP (`curl ifconfig.me` to find it), not `0.0.0.0/0` — widen it later only if the Cloud Run Job specifically needs to reach this VM over the public internet (§5).

2.3 SSH in and prep the host

```
gcloud compute ssh sonarqube-vm --zone=ZONE
```

On the VM:

```
sudo apt-get update
sudo apt-get install -y docker.io docker-compose-plugin
sudo systemctl enable docker
sudo usermod -aG docker "$USER"
# log out and back in (or 'newgrp docker') for the group change to apply

# SonarQube's embedded Elasticsearch refuses to start below these --
# the single most common reason a fresh install crash-loops.
echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
echo "fs.file-max=131072" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

2.4 Bring the stack up

From your own machine, copy the compose file up:

```
gcloud compute scp deploy/gcp/docker-compose.sonarqube.yml \
  sonarqube-vm:~/docker-compose.yml --zone=ZONE
```

Back on the VM, create a **separate** `.env` next to it — this is the VM's own file, unrelated to the agent's `.env` — with a real password:

```
echo "SONAR_DB_PASSWORD=$(openssl rand -base64 24)" > .env
docker compose up -d
docker compose logs -f sonarqube # wait for "SonarQube is operational"
```

2.5 First login, token, and the initial-scan gotcha

Visit `http://EXTERNAL_IP:9000` (get the IP via `gcloud compute instances describe sonarqube-vm --zone=ZONE --format='get(networkInterfaces[0].accessConfigs[0].natIP)'`). Log in with `admin` / `admin` — you'll be forced to set a real password immediately.

Then **My Account** → **Security** → **Generate Token** — this is the value for `SONAR_TOKEN` later.

Easy to miss: the agent's own preflight check requires the target project to already have *at least one* prior analysis under its resolved project key before it will touch it — a key that resolves cleanly from `pom.xml/build.gradle` but has never actually been scanned fails with a clear error rather than silently finding zero issues. Run one manual scan per target repo first:

```
./mvnw sonar:sonar -Dsonar.projectKey=YOUR_KEY \
-Dsonar.host.url=http://EXTERNAL_IP:9000 -Dsonar.token=YOUR_TOKEN
# or
./gradlew sonar -Dsonar.projectKey=YOUR_KEY \
-Dsonar.host.url=http://EXTERNAL_IP:9000 -Dsonar.token=YOUR_TOKEN
```

`CE_EDITION=true` remains the right setting for the agent's `.env` regardless of which SonarQube build you're running — the agent probes the server directly for whether a branch actually exists rather than assuming based on this flag, so getting it slightly "wrong" no longer breaks a run.

2.6 Branch analysis on Community Build

`deploy/gcp/docker-compose.sonarqube.yml` uses `mclarke/sonarqube-with-community-branch-plugin:latest` rather than the stock `sonarqube:lts-community` image — that plugin (source) patches branch analysis, normally a Commercial-edition-only feature, onto the free Community Build. It's a drop-in: the image already sets the required `SONAR_WEB_JAVAADDITIONALOPTS` / `SONAR_CE_JAVAADDITIONALOPTS` `javaagent` flags as its own defaults — nothing else to configure unless you later override those two env vars yourself, in which case you must re-include the `javaagent` line or the plugin silently stops loading.

With this in place, each agent run's analysis actually shows up as its own branch in the SonarQube UI's branch dropdown, instead of always folding back into `main`.

`:latest` tracks whatever SonarQube version the plugin maintainer currently supports; pin a specific tag (check [Docker Hub](#) for what's current) once you want a reproducible deployment rather than a moving target.

2.7 Stopping/starting the VM to save cost

An always-on `e2-standard-2` is the dominant cost of this whole setup. If usage is bursty rather than continuous, stop the VM between sessions — cost scales roughly with uptime (see §7). Two scripts are provided:

```
# once, so you don't need to repeat it below
export SONARQUBE_VM_ZONE=ZONE

deploy/gcp/stop-sonarqube-vm.sh
deploy/gcp/start-sonarqube-vm.sh
```

Docker's `restart: unless-stopped` policy on both compose services means SonarQube and Postgres come back up on their own after a start — no manual `docker compose up` needed — as long as the Docker daemon itself is enabled to start on boot (§2.3).

One gotcha: a VM's default **ephemeral** external IP changes on every stop/start cycle. `start-sonarqube-vm.sh` prints the new IP and flags this, but if `SONAR_BASE_URL` is pointing at the old one, update it — or reserve a static IP once so the address never moves:

```
gcloud compute addresses create sonarqube-vm-ip --region=REGION
gcloud compute instances add-access-config sonarqube-vm --zone=ZONE \
--access-config-name="External NAT" --address=RESERVED_IP
```

Deploy the Agent from the Command Line

3.1 Build and push the image

```
gcloud services enable artifactregistry.googleapis.com run.googleapis.com secretmanager.googleapis.com

gcloud artifacts repositories create sonar-autofix-repo \
  --repository-format=docker \
  --location=REGION

gcloud auth configure-docker REGION-docker.pkg.dev

docker build -t REGION-docker.pkg.dev/PROJECT_ID/sonar-autofix-repo/sonar-autofix-agent:latest .
docker push REGION-docker.pkg.dev/PROJECT_ID/sonar-autofix-repo/sonar-autofix-agent:latest
```

The Dockerfile at the repo root installs Java, Maven, *and* Gradle alongside Python — the agent's adapters shell out to whichever of those the checked-out **target** project actually uses, so both need to be present regardless of which one any single run needs.

3.2 Store secrets

```
printf '%s' 'YOUR_GOOGLE_API_KEY' | gcloud secrets create google-api-key --data-file=-
printf '%s' 'YOUR_SONAR_TOKEN' | gcloud secrets create sonar-token --data-file=-
printf '%s' 'YOUR_GITHUB_TOKEN' | gcloud secrets create github-token --data-file=-
```

3.3 Create the Cloud Run Job

```
gcloud run jobs create sonar-autofix-job \
  --image=REGION-docker.pkg.dev/PROJECT_ID/sonar-autofix-repo/sonar-autofix-agent:latest \
  --region=REGION \
  --set-env-vars=SONAR_BASE_URL=http://SONARQUBE_VM_IP:9000,SOURCE_TYPE=github,GITHUB_REPO=OWNER/REPO,LANGUAGE=java,CE_EDITION=true \
  --set-secrets=GOOGLE_API_KEY=google-api-key:latest,SONAR_TOKEN=sonar-token:latest,GITHUB_TOKEN=github-token:latest \
  --max-retries=0 \
  --task-timeout=3600 \
  --memory=2Gi \
  --cpu=2
```

`--max-retries=0` is deliberate: a failed run has already committed whatever it got done to its own branch — an automatic retry would start a *second*, independent branch on top, not resume the first. Re-run by hand once you've looked at why it failed.

3.4 Run it and watch it

```
gcloud run jobs execute sonar-autofix-job --region=REGION
```

```
gcloud run jobs executions list --job=sonar-autofix-job --region=REGION
gcloud run jobs executions logs read EXECUTION_NAME --region=REGION
```

...or via **Cloud Console** → **Cloud Run** → **Jobs** → **sonar-autofix-job** → **Logs**.

3.5 Redeploying an update

After changing agent code, rebuild, push, then point the job at the new image:

```
docker build -t REGION-docker.pkg.dev/PROJECT_ID/sonar-autofix-repo/sonar-autofix-agent:latest .
docker push REGION-docker.pkg.dev/PROJECT_ID/sonar-autofix-repo/sonar-autofix-agent:latest

gcloud run jobs update sonar-autofix-job \
  --region=REGION \
  --image=REGION-docker.pkg.dev/PROJECT_ID/sonar-autofix-repo/sonar-autofix-agent:latest
```

`update --image=...` only swaps the image — everything else about the job (env vars, secret bindings, resource limits) stays as set in §3.3.

Deploy the Agent via GitHub Actions

4.1 What the workflow does

.github/workflows/deploy-gcp.yml builds this repo's own agent image on every push to `main` that touches agent code, and rolls it out to the Cloud Run Job created in §3.3 — it's the automated version of §3.5. It authenticates via **Workload Identity Federation**: GitHub's own OIDC token is exchanged for short-lived GCP credentials, so there's no long-lived service-account key sitting in GitHub secrets to leak or rotate.

This path still depends on §3.1–3.3 having been done once by hand — `gcloud run jobs update` requires the job to already exist. The workflow never creates the job, only updates its image.

4.2 One-time GCP-side setup (Workload Identity Federation)

```
PROJECT_ID="PROJECT_ID"
REPO="OWNER/REPO"          # this GitHub repo, e.g. acme/remediation-agent
PROJECT_NUMBER="$(gcloud projects describe "$PROJECT_ID" --format='value(projectNumber)')"

# Pool + OIDC provider, scoped to only this repo via attribute-condition
gcloud iam workload-identity-pools create "github-pool" \
  --project="$PROJECT_ID" --location="global" \
  --display-name="GitHub Actions"

gcloud iam workload-identity-pools providers create-oidc "github-provider" \
  --project="$PROJECT_ID" --location="global" \
  --workload-identity-pool="github-pool" \
  --issuer-uri="https://token.actions.githubusercontent.com" \
  --attribute-mapping="google.subject=assertion.sub,attribute.repository=assertion.repository" \
  --attribute-condition="assertion.repository=='${REPO}'"

# Dedicated deploy service account — least privilege: build/push +
# update the one Cloud Run Job, nothing else.
gcloud iam service-accounts create github-deployer \
  --project="$PROJECT_ID" --display-name="GitHub Actions deployer"

SA="github-deployer@${PROJECT_ID}.iam.gserviceaccount.com"

for role in roles/run.developer roles/artifactregistry.writer roles/iam.serviceAccountUser; do
  gcloud projects add-iam-policy-binding "$PROJECT_ID" \
    --member="serviceAccount:${SA}" --role="$role"
done

# Let only this specific repo impersonate that service account
gcloud iam service-accounts add-iam-policy-binding "$SA" \
  --project="$PROJECT_ID" \
  --role="roles/iam.workloadIdentityUser" \
  --member="principalSet://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityPools/github-pool/attribute.repository/${REPO}"
```

4.3 One-time GitHub-side setup

In the repo's **Settings** → **Secrets and variables** → **Actions** → **Variables** tab — these are **Variables**, not Secrets, since a WIF provider path and a service-account email aren't sensitive on their own:

Variable	Value
GCP_PROJECT_ID	PROJECT_ID
GCP_REGION	the region used in §3
GCP_WIF_PROVIDER	projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/github-pool/providers/github-provider
GCP_DEPLOY_SERVICE_ACCOUNT	github-deployer@PROJECT_ID.iam.gserviceaccount.com

4.4 Triggering a deploy

Push to `main`, or run it manually via **Actions** → **Deploy agent to Cloud Run** → **Run workflow**. It builds, pushes, and updates the job in place. The job's env vars and secret bindings (`SONAR_BASE_URL`, `GITHUB_REPO`, the Secret Manager references, etc.) are untouched — only the image changes.

Scope, on purpose: executing the job — actually running the agent against a target repo — stays a separate, deliberate action: either `gcloud run jobs execute` by hand, or the optional Cloud Scheduler wiring in §6. This workflow only keeps the job's *image* current, matching the same "no silent auto-retry" philosophy as `--max-retries=0` in §3.3.

STEP 5

Networking: Cloud Run Reaching the SonarQube VM

Cloud Run Jobs, by default, egress through Google's shared internet IPs — not a fixed address you can allowlist on the VM's firewall. Two options:

- **Simple (pilot-only):** open the VM's firewall rule to `0.0.0.0/0` on port 9000. Fast to set up; the tradeoff is SonarQube's login page becomes reachable by anyone on the internet — still gated by the admin password and `SONAR_TOKEN` for API access, but a weak admin password is now a real exposure, not a theoretical one.
- **Recommended:** create a [Serverless VPC Access connector](#) in the same VPC as the SonarQube VM, attach it to the Cloud Run Job (`--vpc-connector` , `--vpc-egress=all-traffic`), and point `SONAR_BASE_URL` at the VM's **internal** IP instead. The firewall rule then only needs to allow the VPC's own internal range, not the public internet.

Start with the simple path to get the first end-to-end run working, then move to the VPC connector before this touches anything you'd call production.

Scheduled Runs

```
gcloud iam service-accounts create sonar-autofix-invoker

gcloud run jobs add-iam-policy-binding sonar-autofix-job \
  --region=REGION \
  --member="serviceAccount:sonar-autofix-invoker@PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/run.invoker"

gcloud scheduler jobs create http sonar-autofix-nightly \
  --schedule="0 2 * * *" \
  --uri="https://REGION-run.googleapis.com/apis/run.googleapis.com/v1/namespaces/PROJECT_ID/jobs/sonar-autofix-job:run" \
  --http-method=POST \
  --oauth-service-account-email="sonar-autofix-invoker@PROJECT_ID.iam.gserviceaccount.com"
```

This is genuinely optional for a first pass — get one manual `gcloud run jobs execute` working end to end before automating *when* it runs.

REFERENCE

What This Costs

List prices, `us-central1` -ish, no committed-use discounts — treat this as a ballpark, not a quote. The [pricing calculator](#) gives an exact figure for your region and usage pattern.

Component	Driver	Approx. cost
SonarQube VM (<code>e2-standard-2</code> , 24/7)	~\$0.067/hr × 730 hr	~\$49/mo
Boot disk (30 GB, pd-balanced)	~\$0.10/GB-mo	~\$3/mo
Cloud Run Job (agent, 2 vCPU/2Gi, ~20 min/run)	~\$0.06 compute/run	free tier covers ~75 runs/mo before you pay anything
Artifact Registry (image storage)	~1–1.5 GB image, \$0.10/GB-mo	~\$0.10–0.15/mo
Secret Manager	3 secrets, 1 version each	\$0 (free tier covers 6 active versions/mo)
Egress / misc	API calls: agent ↔ SonarQube ↔ GitHub	usually cents/mo
<i>Optional</i> : VPC connector (\$5, recommended route)	2 min instances × ~\$0.01/hr	+~\$15–30/mo

Bottom line: roughly \$52–55/month, almost entirely the always-on SonarQube VM. The Cloud Run Job side is close to free at pilot usage.

Biggest lever if cost matters: SonarQube doesn't need to run 24/7. Stop the VM between sessions with `deploy/gcp/stop-sonarqube-vm.sh` (§2.7) — cost scales down roughly with uptime, e.g. ~8 hrs/day cuts the \$49 to more like \$16–20/month.

SUMMARY

Recap — Automated vs. Still Manual

- **Automated by this setup:** build/push the image (either path), rolling the new image onto the job, the run itself once triggered, secret injection, logs.
- **Still manual, by design:** re-running after a failure (no auto-retry — §3.3), rotating the SonarQube admin password, the one-time initial scan per target project (§2.5), and creating the Cloud Run Job itself the first time (§3.3) — the GitHub Actions path only ever updates an existing job.

Files Referenced in This Guide

File	Purpose
Dockerfile	Builds the agent image — Java, Maven, Gradle, Python, git identity config.
.dockerignore	Keeps secrets and dev artifacts out of the image build context.
deploy/gcp/docker-compose.sonarqube.yml	SonarQube (with the community branch plugin) + Postgres, for the Compute Engine VM.
deploy/gcp/start-sonarqube-vm.sh deploy/gcp/stop-sonarqube-vm.sh	Stop/start the SonarQube VM between sessions to save cost.
.github/workflows/deploy-gcp.yml	CI/CD: builds and rolls the agent image onto the existing Cloud Run Job.
docs/GCP_DEPLOYMENT.md	The full source guide this document was generated from.